

Conception et développement d'un système de suivi d'activité utilisateur pour visualisations web

Composant de suivi du regard de l'*Activity Tracker*

Projet PIR, Laboratoire i3S, Université Côte d'Azur

Encadrante : Aline Menin

29 juin 2026

Résumé

Ce rapport décrit le prototype du composant de **suivi du regard** (*eye-tracking*) d'un *Activity Tracker* destiné à l'étude des processus de raisonnement analytique sur des visualisations de données web. Le prototype repose sur **WebGazer.js** pour estimer la direction du regard à partir d'une webcam standard, sans matériel spécialisé. Il fournit : un module de capture, une calibration avancée (correction spatiale continue, compensation des mouvements de tête, validation par leave-one-out), un post-traitement extrayant **fixations** et **saccades** (algorithmes I-DT et I-VT), et un **format de journalisation structuré** exporté en JSON, JSON-LD et CSV pour intégration ultérieure dans un graphe de connaissances. Deux moteurs de regard interchangeables sont proposés au choix de l'utilisateur (WebGazer.js et un moteur maison fondé sur MediaPipe FaceLandmarker). La validation logicielle s'appuie sur 506 tests unitaires automatisés. Ce document constitue une première version : la section d'évaluation utilisateur (3–5 participants) présente le protocole et un cadre d'analyse à compléter par les données réelles.

Table des matières

1	Introduction et contexte	3
2	Étude des solutions open-source	3
3	Module de capture du regard	4
3.1	Architecture	4
3.2	Capture	4
4	Calibration et précision	4
4.1	Procédure	4
4.2	Correction spatiale	4
4.3	Score honnête par validation croisée	5
4.4	Lissage temporel et compensation de tête	5
4.5	Comparaison des conditions de capture	5
5	Second moteur : MediaPipe FaceLandmarker	6
6	Extraction des fixations et saccades	6
7	Format de journalisation structuré	7
7.1	Structure	7
7.2	Choix de conception	7
7.3	Richesse contextuelle de chaque point de regard	7
7.4	Export orienté graphe de connaissances (JSON-LD)	8
8	Évaluation avec participants	8
8.1	Protocole : session de test guidée et automatique	8
9	Visualisation et consultation des sessions	9
10	Validation logicielle	9
11	Limitations et perspectives	9
12	Conclusion	10

1 Introduction et contexte

Comprendre comment les utilisateurs explorent visuellement les visualisations de données est essentiel pour étudier le raisonnement analytique. En recherche en visualisation d'information, les journaux d'interaction et le suivi du regard sont couramment combinés pour analyser le comportement et l'attention visuelle.

Les progrès du suivi du regard *dans le navigateur*, en particulier WebGazer.js [1], permettent d'estimer le regard depuis une webcam standard, ce qui facilite les études en ligne à grande échelle. Ce projet s'inscrit dans un effort plus large visant à construire un *Activity Tracker* collectant des données multimodales (interactions, regard, verbalisations) pour étudier les processus analytiques et soutenir la conception de visualisations adaptatives, les données étant à terme intégrées dans un **graphe de connaissances**.

Objectif. Prototyper le composant de suivi du regard dans un environnement de visualisation web. Les tâches sont : étudier les solutions open-source ; implémenter la capture ; extraire fixations et saccades ; définir un format de journalisation ; tester avec 3–5 participants.

2 Étude des solutions open-source

Plusieurs bibliothèques de suivi du regard par webcam existent. Le tableau 1 résume les principales options évaluées.

Critère	WebGazer.js	MediaPipe FaceLandmarker	GazeCloudAPI
Exécution	100 % navigateur	100 % navigateur (WASM/GPU)	Service cloud
Matériel	Webcam standard	Webcam standard	Webcam standard
Licence	Open-source (GPLv3)	Open-source (Apache)	Propriétaire
Vie privée	Local	Local	Vidéo au cloud
Sortie	Point écran direct	Landmarks + pose (mapping à coder)	Point écran
Calibration	Intégrée	À implémenter (faite ici)	Intégrée
Maturité	Élevée, citée en recherche	Modèle récent, précis	Moyenne

TABLE 1 – Comparaison des solutions de suivi du regard par webcam.

Choix : les deux moteurs locaux. GazeCloudAPI est écarté (vie privée : il transmet la vidéo à un tiers). Deux moteurs *locaux* sont retenus et proposés au choix de l'utilisateur via une page d'accueil :

- **WebGazer.js** : solution clé-en-main, mapping regard→écran intégré, reproductible et citée en recherche ; idéal pour un prototype rapide.
- **MediaPipe FaceLandmarker** : détection d'iris et pose de tête plus précises, mais ne fournit pas de point de regard : tout le mapping a été développé (cf. §5).

Proposer les deux permet de comparer empiriquement précision et robustesse sur le même protocole et le même format de données.

3 Module de capture du regard

3.1 Architecture

Le prototype est une application web statique (HTML + JavaScript, sans étape de *build*), organisée en modules indépendants exposés comme objets globaux (tableau 2).

Fichier	Global	Rôle
<code>gaze-capture.js</code>	<code>GazeCapture</code>	Démarrage/arrêt de WebGazer, émission des points.
<code>calibration.js</code>	<code>Calibration</code>	Calibration, correction spatiale, lissage, détection fixations/saccades.
<code>gaze-logger.js</code>	<code>GazeLogger</code>	Journalisation et export structuré (JSON / JSON-LD).

TABLE 2 – Modules principaux et API publiques.

3.2 Capture

Le module `GazeCapture` vérifie la disponibilité de la webcam via `getUserMedia`, initialise WebGazer (régression *ridge*, traqueur TFFacemesh), puis émet à chaque trame un point $\{x, y, \text{timestamp}\}$ aux abonnés. Deux précautions améliorent la qualité du modèle : la suppression des écouteurs souris (`removeEventListener`), car le mouvement de souris pollue l'entraînement, et la contrainte explicite de résolution caméra.

4 Calibration et précision

La **qualité de calibration** est le principal point faible du suivi du regard par webcam ; un effort particulier lui a donc été consacré.

Note de méthode. Cette section (calibration et précision) a été conçue en nous appuyant sur des **recherches bibliographiques** et sur l'aide d'une **intelligence artificielle** pour choisir et mettre au point les méthodes mathématiques. Les autres parties du rapport ne reposent pas sur ces outils.

4.1 Procédure

On affiche une grille de **25 points** (5×5 , plus serrée sur les bords). À chaque point, l'utilisateur clique plusieurs fois : on enregistre à chaque clic où il regarde et où se trouve réellement le point. Une autre grille de **9 points**, qui n'a pas servi à l'apprentissage, permet de vérifier la précision. Avant de commencer, un écran de réglage contrôle la luminosité, l'équilibre de l'éclairage et la distance au visage (≈ 60 cm) à partir de l'image de la webcam.

4.2 Correction spatiale

Même après calibration, l'erreur n'est pas la même partout sur l'écran : WebGazer rend l'erreur *petite en moyenne*, mais il reste des décalages **qui dépendent de l'endroit** regardé, typiquement plus forts dans les coins et sur les bords, là où l'œil est mal vu par la caméra. Comme

ce décalage *change de sens et de taille selon la zone de l'écran*, on ne peut pas le corriger avec un seul réglage valable partout. On estime donc une **carte de correction** qui, pour n'importe quel point p de l'écran, indique le petit vecteur à retirer pour viser juste.

Cette carte est construite à partir des 9 points de vérification, pour lesquels on connaît l'erreur mesurée e_i (écart entre le regard prédit et le vrai point). Pour corriger un point p quelconque, on fait une **moyenne des erreurs voisines**, en donnant plus de poids aux points proches (méthode **LOWESS**) :

$$\hat{e}(p) = \frac{\sum_i w_i e_i}{\sum_i w_i}, \quad w_i = \left(1 - \left(\frac{d_i}{h}\right)^3\right)^3 \text{ si } d_i < h, \quad w_i = 0 \text{ sinon,}$$

où d_i est la distance entre p et le point de vérification i , et h un rayon fixé (jusqu'où on regarde autour). Cette formule a trois qualités utiles :

- **Elle reste locale.** Le poids w_i diminue avec la distance et tombe à zéro au-delà du rayon h : seuls les points proches comptent. La correction suit donc les variations d'une zone à l'autre, au lieu de tout mélanger.
- **Elle est douce.** Le poids vaut 1 juste sur le point, puis redescend progressivement jusqu'à 0 au bord du rayon, sans à-coup. La correction change donc en douceur quand le regard se déplace, sans saut visible du curseur.
- **Elle résiste au bruit.** Comme on fait une moyenne de plusieurs mesures voisines au lieu de coller exactement à chacune, une mesure isolée et fautive ne dérègle pas la carte.

Le rayon h est un compromis : trop petit, la correction colle trop aux mesures et devient instable ; trop grand, elle lisse tout et perd en finesse. Quand aucun point de vérification ne tombe dans le rayon, on bascule sur une moyenne pondérée par l'inverse de la distance (IDW), pour rester défini partout, même en dehors de la zone calibrée.

Une erreur a été repérée et corrigée : une version précédente appliquait *deux* corrections à la suite (IDW puis LOWESS) qui estimaient *la même* erreur. Corriger deux fois revenait à trop corriger, et le curseur partait dans l'autre sens (**sur-correction**). On n'applique désormais qu'une seule correction, dont la qualité est vérifiée honnêtement (§4.3).

4.3 Score honnête par validation croisée

Mesurer la précision sur les points qui ont servi à construire la correction donne un résultat trop optimiste. On calcule donc l'erreur autrement, par **leave-one-out** : pour chaque point de vérification, on reconstruit la carte de correction *sans* ce point, puis on mesure l'erreur restante sur ce point mis de côté. C'est ce chiffre, plus réaliste, qui est affiché.

4.4 Lissage temporel et compensation de tête

Pendant l'utilisation, un filtre **One Euro** [2] atténue le petit tremblement du curseur au repos, sans ajouter de retard quand le regard se déplace vite. WebGazer supposant que la tête ne bouge pas, on ajoute aussi une **compensation des mouvements de tête** : on retient la position de référence du visage, et tout déplacement de la tête décale le curseur d'une quantité proportionnelle et limitée, ce qui réduit la dérive sur les longues sessions.

4.5 Comparaison des conditions de capture

À partir des sessions archivées dans `data/`, nous comparons l'erreur moyenne de calibration (en pixels) selon quatre conditions de capture. Les valeurs indisponibles sont notées *N.D.* : dans ces configurations, le visage n'a pas pu être détecté de manière suffisamment fiable pour produire une mesure exploitable.

Voici les différentes conditions de capture :

- **Situation par défaut** : webcam placée au-dessus de l'écran, luminosité moyenne, sans lunettes.
- **Lumière claire** : une meilleure visibilité des traits du visage améliore la détection des repères et réduit l'erreur de calibration.
- **Lunettes** : les reflets sur les verres peuvent perturber la détection des yeux et augmenter l'erreur.
- **Caméra sous l'écran** : cette configuration se rapproche de celle des eye-trackers classiques, généralement positionnés sous l'écran.

Condition	Mediapipe (base)	Mediapipe point mouvant	WebGazer
Situation par défaut	165.2 px	139.4 px	<i>N.D.</i>
Lumière claire	132.2 px	219.0 px	7086.5 px
Port de lunettes	166.8 px	183.8 px	<i>N.D.</i>
Caméra sous l'écran	113.1 px	225.6 px	7359.3 px

TABLE 3 – Comparaison des conditions de capture à partir des sessions du dossier `data/`.

5 Second moteur : MediaPipe FaceLandmarker

Un second moteur, alternatif et interchangeable avec WebGazer, a été développé sur **MediaPipe FaceLandmarker**. Contrairement à WebGazer, MediaPipe ne fournit pas un point de regard à l'écran mais 478 points caractéristiques 3D (iris inclus) et une matrice de pose de tête. Toute la chaîne de régression *caractéristiques*→*écran* a donc été construite :

1. **Extraction de caractéristiques** : position relative de l'iris dans chaque orbite, exprimée dans un *repère frontal canonique* obtenu en annulant la rotation de tête (projection par $R^\top$) ; angles d'Euler de la pose (yaw/pitch/roll) ; échelle inter-oculaire (proxy de distance) ; termes polynomiaux et blendshapes oculaires.
2. **Régression ridge** : apprentissage par moindres carrés régularisés sur caractéristiques *standardisées*, avec pondération des échantillons par leur qualité et choix automatique du paramètre λ par validation croisée k-fold.
3. **Correction résiduelle et lissage** : champ de correction (IDW/LOWESS) appris sur la validation, filtre One Euro temps réel, et *apprentissage continu* (recalibration incrémentale par clics en cours d'usage).

Ce moteur offre une compensation des mouvements de tête *native* (la pose fait partie des caractéristiques) et une cadence élevée grâce à l'inférence GPU synchronisée sur les images de la webcam. Les deux moteurs partagent la même chaîne de post-traitement, le même journal et le même format d'export, ce qui rend leur comparaison directe sur le même protocole.

Deux modes de calibration. Outre la calibration classique par clics (grille de 25 points), un mode **calibration par poursuite** (*smooth pursuit*) a été ajouté pour MediaPipe : une cible mobile parcourt l'écran en serpentins et l'utilisateur la suit du regard, sans cliquer. Ce mode collecte beaucoup plus d'échantillons (plusieurs centaines) le long d'une trajectoire continue, ce qui produit souvent un modèle de régression plus stable.

6 Extraction des fixations et saccades

Deux algorithmes classiques de l'analyse du regard [3] sont implémentés.

Fixations : I-DT (Dispersion-Threshold). Une fenêtre temporelle glissante regroupe les points dont la dispersion spatiale $(\max_x - \min_x) + (\max_y - \min_y)$ reste sous un seuil pendant une durée minimale. L'implémentation utilise des *deque*s monotones pour calculer la dispersion en $O(n)$ amorti ; le traitement de 10 000 points s'exécute en moins de 50 ms.

Saccades : I-VT (Velocity-Threshold). Les transitions dont la vitesse instantanée dépasse un seuil sont marquées comme saccades, caractérisées par leur amplitude et leur vitesse maximale. Une fonction `linkEvents` produit une chronologie alternant fixations et saccades.

Chaque fixation peut être associée à une **zone d'intérêt** (AOI) de la visualisation regardée, base de l'analyse attentionnelle.

7 Format de journalisation structuré

7.1 Structure

Une session exportée comporte cinq sections (listing 1) : métadonnées `session`, points bruts `raw_gaze_data`, événements `events` (fixations/saccades), `aoi_hits`, et `interactions` multimodales. Le format est décrit par un **JSON Schema** (draft-07) qui le rend validable et auto-documenté.

Listing 1 – Structure d'une session exportée (extrait).

```

1 {
2   "session": {
3     "id": "uuid", "format_version": "1.1.0",
4     "participant_id": "P01", "clock_origin_ms": 12345.6,
5     "calibration_score": { "mean_error_px": 92.4 },
6     "config_snapshot": { "lowess_bandwidth": 0.45, ... }
7   },
8   "raw_gaze_data": [ { "x": 812, "y": 437, "t_rel_ms": 4502.1 }, ... ],
9   "events": [ { "type": "fixation", "duration": 280,
10                "details": { "x": 810, "y": 440 } }, ... ],
11   "aoi_hits": [ { "aoi_id": "bar-q3", "event_index": 12 }, ... ],
12   "interactions": [ { "type": "tab_change", "t_rel_ms": 8100.0 }, ... ]
13 }
```

7.2 Choix de conception

- **Double horloge.** Chaque enregistrement porte un `timestamp epoch` (lisible) *et* un `t_rel_ms` issu de `performance.now()`, horloge **monotone** indispensable au calcul fiable des durées et vitesses.
- **Reproductibilité.** `config_snapshot` fige les paramètres de filtrage/correction effectifs, permettant de rejouer ou comparer deux sessions.
- **Multimodalité.** La section `interactions` journalise clics, changements d'onglet et entrées du regard dans une AOI, conformément à l'objectif d'*Activity Tracker*.

7.3 Richesse contextuelle de chaque point de regard

Au-delà des coordonnées, chaque point de regard est enrichi de tout le contexte permettant de décrire *ce que l'utilisateur regardait* :

- **Confiance** de la prédiction ($\in [0, 1]$) et **coordonnées brutes** avant correction, en plus des coordonnées corrigées.
- **Module source** (`webgazer` ou `mediapipe`) sur chaque entrée, rendant chaque module clairement identifiable dans le journal.

- **Descripteur DOM** de l’objet observé : balise, identifiant, classes, *type sémantique* (barre, axe, point, légende, courbe, étiquette), boîte englobante, texte et attributs `data-*`, sélecteur CSS. On sait ainsi si le regard portait sur une barre, un axe, etc.
- **État de la visualisation** : vue active, jeu de données, niveau de zoom, filtres actifs, sélection et AOI courante. Un historique `viz_states` enregistre en plus chaque changement d’état (zoom, filtre par trimestre, onglet).

Chaque point porte en outre la **luminosité ambiante** (lux, mesurée par l’API *Ambient-LightSensor* ou estimée depuis la webcam) et, pendant un test guidé, le **contexte du stimulus** (`test_case_id`, `target_aoi_id`). La session enregistre les informations participant (`first_name`, `last_name`, `glasses`, `engine`). Le format est versionné (v1.3).

Trois formats d’export sont fournis : **JSON** (structuré), **JSON-LD** (graphe de connaissances) et **CSV** (analyse tabulaire directe sous pandas/R/Excel).

7.4 Export orienté graphe de connaissances (JSON-LD)

Un second export en **JSON-LD** représente la session comme un graphe de nœuds typés sous un vocabulaire `wga` : (`wga:Session`, `wga:Fixation`, `wga:Saccade`, `wga:AOIHit`, `wga:Interaction`). Ce format se charge directement dans un triple store RDF, anticipant l’intégration au graphe de connaissances de l’*Activity Tracker*.

8 Évaluation avec participants

8.1 Protocole : session de test guidée et automatique

Les deux pages moteurs (`index-webgazer.html` et `index-mediapipe.html`) implémentent le même protocole **entièrement guidé et automatique**, sans aucune action de l’utilisateur pendant le test :

1. **Formulaire participant** : prénom, nom, port de lunettes.
2. **Calibration** (par clics ou, pour MediaPipe, par poursuite) puis lancement *automatique* du scénario.
3. **Scénario** : enchaînement automatique des trois visualisations (page 1 bar chart, page 2 line chart, page 3 scatter). Sur chaque page, **trois cercles rouges** « regardez ici » apparaissent successivement (≈ 3 s chacun) sur des AOI réelles distinctes, puis une phase d’**exploration libre** de 15s, avant le passage automatique à la page suivante. Chaque point est étiqueté avec le stimulus, l’AOI cible et sa position écran (`test_case_id`, `target_aoi_id`, `target_x/y`).
4. **Résultats** : post-traitement, enregistrement de la session et affichage de l’écran de résultats détaillés (analyse + parcours).

Un **retour visage permanent** (visage détecté, distance, luminosité) reste affiché en continu, y compris pendant le test, pour signaler tout problème de captation. Chaque session produit **un fichier JSON par personne** et est persistée localement pour reconsultation.

Mesures collectées. Erreur de calibration (px), luminosité (lux), confiance moyenne, nombre de fixations/saccades, hits AOI, et, grâce à l’étiquetage des stimuli, la correspondance entre l’AOI ciblée et l’objet réellement regardé.

À compléter. Les sessions réelles (3–5 participants) seront collectées via cette page et consultées dans le visualiseur (§9) : tableau récapitulatif par participant (nom, date, lunettes, lux, erreur de calibration), distribution des erreurs et synthèse qualitative.

9 Visualisation et consultation des sessions

Une sous-page `session-viewer.html` permet de **consulter et comparer** les sessions enregistrées sans relancer de test. Les sessions étant persistées localement (le projet étant statique, `localStorage` sert de base de données ; un export/import fichier permet l’archivage et le transfert), la page offre :

- un **tableau triable et filtrable** (par moteur, par port de lunettes) de toutes les sessions (date, participant, moteur, erreur de calibration, lux, confiance, nombre de points/fixations/-saccades) ;
- un **écran de résultats détaillés** en deux onglets : une *analyse par stimulus* (page regardée, AOI ciblée, score « sur la cible », durée) et un *parcours du regard* montrant les **trois visualisations re-dessinées** avec, par-dessus chacune, la **heatmap** de densité (vert→orange→rouge) et le scanpath des fixations propres à cette page ;
- une **comparaison** entre deux sessions ou deux groupes (p.ex. toutes les sessions WebGazer contre toutes les MediaPipe), présentée sous forme d’un **bilan écrit en français** (quel moteur est meilleur et pourquoi) suivi d’un tableau détaillé métrique par métrique (valeur, écart, gagnant) ;
- l’**export** par session (JSON, CSV) et global.

Score « sur la cible » continu. Pendant le test, la position écran du stimulus (cercle rouge) est journalisée avec chaque point de regard. Le score d’une cible est la moyenne, sur ses points, d’une fonction de proximité valant 100 % si le regard est dans un rayon serré autour du centre et décroissant continûment avec la distance — bien plus informatif qu’un simple « dans/hors de l’AOI ».

10 Validation logicielle

La logique métier (algorithmes I-DT/I-VT, filtres, correction spatiale, format de log) est couverte par des **tests unitaires** automatisés (Node, sans webcam), exécutables par `npm test`. Ils vérifient notamment : la correction unique (non sur-correctée), la validation leave-one-out, le lissage One Euro, la robustesse des détecteurs sur cas limites, et la performance (<50 ms sur 10 000 points).

11 Limitations et perspectives

Limitations. La précision reste dépendante de l’éclairage, de la qualité de la webcam et de l’immobilité de la tête, limites intrinsèques au suivi du regard par webcam. La fréquence d’échantillonnage varie selon la machine (typiquement 10–30 Hz). Une dérive temporelle subsiste sur les longues sessions, seulement atténuée par la compensation de tête.

Perspectives.

- Réaliser les sessions réelles avec participants et consolider l’analyse quantitative.
- Étendre la multimodalité aux **verbalisations** (think-aloud) prévues par l’*Activity Tracker*.
- Définir et publier le vocabulaire `wga:` et charger les exports JSON-LD dans le graphe de connaissances cible.
- Étalonner empiriquement le gain de compensation de tête sur plusieurs configurations matérielles.

12 Conclusion

Le prototype répond aux objectifs fixés : une solution open-source a été choisie et justifiée, la capture, le post-traitement (fixations/saccades) et un format de journalisation structuré (JSON et JSON-LD) sont opérationnels et testés. La calibration a fait l'objet d'améliorations dépassant le cadre initial (correction spatiale continue, score honnête par leave-one-out, compensation de tête). Il reste à mener l'évaluation utilisateur pour quantifier en conditions réelles la stabilité de la capture et les limites pratiques de l'approche.

Références

- [1] A. Papoutsaki *et al.*, *WebGazer : Scalable Webcam Eye Tracking Using User Interactions*, IJCAI, 2016.
- [2] G. Casiez, N. Roussel, D. Vogel, *1 € Filter : A Simple Speed-based Low-pass Filter for Noisy Input in Interactive Systems*, ACM CHI, 2012.
- [3] D. D. Salvucci, J. H. Goldberg, *Identifying Fixations and Saccades in Eye-Tracking Protocols*, ETRA, 2000.